

Order-Flow and Imbalance Features

Formulas, estimation settings and reference implementations
from ten source papers, fitted dynamically on data

Compiled from the source PDFs

September 2026

Abstract

This document states how order-flow imbalance (OFI) and related microstructure features are constructed, exactly as each source paper defines them, and gives reference implementations that are fitted on rolling windows with point-in-time inputs only. Every formula is transcribed from the source PDF, with evident typographical errors in the published versions corrected here. Where a paper leaves a step genuinely unstated, the document says so and names the choice the implementer has to make. Section 1 fixes notation; section 2 gives the shared fitting protocol; sections 3–8 cover the five order-book constructions and the trade-only construction; sections 9–10 cover feature construction by clustering and the two prerequisites for applying any of this to corporate bonds. Section 12 maps each result back to its paper.

Contents

1	Data model, clocks and notation	2
2	Fitting protocol	3
3	Multi-level order-flow imbalance (MLOFI)	4
3.1	Definition	4
3.2	Model and estimation	4
3.3	Results	5
4	Integrated OFI and cross-impact	5
4.1	Definition	6
4.2	Models	6
4.3	Results	7
5	Decomposed OFI by event type	8
5.1	Definition	8
5.2	Models and estimation	9
5.3	Results	9
6	Generalised OFI for snapshot data	11
6.1	The problem and the fix	11
6.2	Results	11

7 OFI as a shock: horizon choice and screening	12
7.1 Four metrics, stated in one place	12
7.2 The shock model	13
7.3 A screening test worth reusing	13
8 Generalised imbalance from trades alone	14
8.1 Definition	14
8.2 What to measure on data	15
8.3 The simulator, and why it matters for bonds	15
9 Building flow features by clustering	16
9.1 The 16 features	16
9.2 Normalisation, reduction, clustering	17
10 Prerequisites for corporate bonds	18
10.1 Signing trades	19
10.2 Correcting for bid-ask noise	19
11 Supporting results	20
12 Source map	21

1 Data model, clocks and notation

A *book event* is any message that changes the visible limit order book: a limit order arrival, a cancellation or deletion, or an execution. Events are indexed by n in arrival order with timestamps τ_n , and

$$N(t) = \max\{n : \tau_n \leq t\}$$

is the index of the last event no later than t . A *bucket* of length h ending at t is $(t - h, t]$, containing events $n = N(t - h) + 1, \dots, N(t)$, and $\Delta N(t) = N(t) - N(t - h)$ counts them.

At level m after event n , write $P_n^{m,b}$, $P_n^{m,a}$ for the bid and ask prices and $q_n^{m,b}$, $q_n^{m,a}$ for the queue sizes; level $m = 1$ is the touch. All book quantities are measured *immediately after* the event is applied. Only populated levels count, so level $m + 1$ need not be one tick beyond level m . For trade data without a book, $\varepsilon_t \in \{-1, +1\}$ is the trade sign and q_t the traded size. The mid-price is $P_t = (P_t^{1,b} + P_t^{1,a})/2$.

Three conventions decide whether an implementation is correct:

1. **Bucket alignment.** A contemporaneous regression puts feature and return on the *same* bucket. A forecasting regression puts the feature on $(t - h, t]$ and the return on $(t, t + f]$. Mixing them produces a look-ahead result that looks excellent.
2. **Sign convention.** Most papers here use +1 for buyer-initiated flow. Gould et al. (section 11) use the opposite. Check before combining series from different sources.
3. **Timestamp units.** Event times and horizons must be compared in one unit. Casting a timestamp series in a resolution-dependent way while comparing it against a fixed-unit scalar silently empties every window.

2 Fitting protocol

All features below share one estimation pattern, kept separate from the feature definitions so a new feature can be dropped in without touching the estimator.

Rolling windows. Parameters are estimated on a trailing window of W observations ending at t and applied only to later observations. The source papers use short windows: 30 minutes of 10-second buckets (180 observations) contemporaneously, 30 minutes of 1-minute buckets (30 observations) for forecasting, re-estimated every minute.

Regularisation. Multi-level OFI regressors are correlated across levels by construction, and the event-type decomposition triples the regressor count on the same window. Ordinary least squares over-fits in both cases: Xu et al. report out-of-sample error *rising* beyond about five levels under OLS but not under ridge, and Sitaru et al. report out-of-sample R^2 below -100% under OLS at six levels or more. The papers use ridge with cross-validated penalty (Xu), LASSO with cross-validated penalty (Cont, Sitaru, Hu), or principal components.

Evaluation. In-sample fit is adjusted R^2 . Out-of-sample fit is computed on the next disjoint window. For short-horizon *forecasts*, out-of-sample R^2 in this literature is routinely negative even where the economic evaluation is strongly positive, so a trading-rule evaluation is reported alongside it (sections 4 and 5).

```
import numpy as np
import pandas as pd
from sklearn.linear_model import Ridge, LassoCV, RidgeCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

def rolling_fit_predict(X, y, window, make_model, step=1):
    """Fit on a trailing window, predict forward. Point-in-time by construction.

    Row  $t$  is predicted by a model fitted on rows  $[t - \text{window}, t - 1]$  only.
    """
    X, y = X.sort_index(), y.reindex(X.sort_index().index)
    out = []
    for end in range(window, len(X), step):
        xtr, ytr = X.iloc[end - window:end], y.iloc[end - window:end]
        ok = xtr.notna().all(axis=1) & ytr.notna()
        if ok.sum() < max(10, X.shape[1] + 2):
            continue # not enough rows to identify the fit
        model = make_model().fit(xtr[ok], ytr[ok])
        te = slice(end, min(end + step, len(X)))
        out.append(pd.Series(model.predict(X.iloc[te].fillna(0.0)),
                             index=X.index[te], name="y_hat"))
    return pd.concat(out) if out else pd.Series(dtype=float, name="y_hat")

def oos_r2(y_true, y_hat, benchmark=0.0):
    """Out-of-sample  $R^2$  against a constant benchmark.

    NOTE: none of the source papers writes this formula down. This is the
    standard choice (zero benchmark); state it explicitly in any writeup.
    """
```

```

y_true, y_hat = y_true.align(y_hat, join="inner")
ss_res = float(((y_true - y_hat) ** 2).sum())
ss_tot = float(((y_true - benchmark) ** 2).sum())
return 1.0 - ss_res / ss_tot if ss_tot > 0 else np.nan

```

Listing 1: Shared rolling-fit harness. The estimator is injected, so the same code serves ridge, LASSO and principal-component variants.

3 Multi-level order-flow imbalance (MLOFI)

Source. Xu, Gould & Howison (2020), *Multi-Level Order-Flow Imbalance in a Limit Order Book*, Market Microstructure and Liquidity 4(3&4).

3.1 Definition

Write $r^m(\tau_n)$ for the total size at the level- m bid and $q^m(\tau_n)$ for the total size at the level- m ask. The bid- and ask-side changes are

$$\Delta W^m(\tau_n) = \begin{cases} r^m(\tau_n), & b^m(\tau_n) > b^m(\tau_{n-1}), \\ r^m(\tau_n) - r^m(\tau_{n-1}), & b^m(\tau_n) = b^m(\tau_{n-1}), \\ -r^m(\tau_{n-1}), & b^m(\tau_n) < b^m(\tau_{n-1}), \end{cases} \quad (1)$$

$$\Delta V^m(\tau_n) = \begin{cases} -q^m(\tau_{n-1}), & a^m(\tau_n) > a^m(\tau_{n-1}), \\ q^m(\tau_n) - q^m(\tau_{n-1}), & a^m(\tau_n) = a^m(\tau_{n-1}), \\ q^m(\tau_n), & a^m(\tau_n) < a^m(\tau_{n-1}), \end{cases} \quad (2)$$

and the level- m contribution and its aggregate are

$$e^m(\tau_n) = \Delta W^m(\tau_n) - \Delta V^m(\tau_n), \quad \text{MLOFI}^m(t_{k-1}, t_k) = \sum_{\{n: t_{k-1} < \tau_n \leq t_k\}} e^m(\tau_n). \quad (3)$$

Setting $M = 1$ reproduces the original best-level OFI of Cont et al. (2014) exactly. Hidden orders are excluded.

No normalisation. This paper applies none at all — not by depth, event count, spread or volatility. MLOFI is a raw signed share count, and the paper lists rescaling as future work. Every other construction in this document divides by something, which matters when comparing coefficient magnitudes across them.

3.2 Model and estimation

$$\Delta P(t_{k-1}, t_k) = \alpha + \sum_{m=1}^M \beta_m \text{MLOFI}^m(t_{k-1}, t_k) + \varepsilon, \quad M = 10, \quad (4)$$

with $\Delta P(t, t + \Delta t) = P(t + \Delta t) - P(t)$ the contemporaneous mid-price change. Ridge minimises $\|y - X\beta\|_2^2 + \lambda\|\beta\|_2^2$ with λ chosen by 5-fold cross-validation over 50 logarithmically spaced candidates in $[10^{-5}, 10^5]$. The fitted penalties differ by two orders of magnitude across stocks: AMZN 139.0, TSLA 54.3, NFLX 33.9, ORCL 3.2, CSCO 1.3, MU 2.0. In the paper’s ridge formulation the design matrix carries a leading column of ones, so the intercept sits inside the penalised vector; no standardisation of regressors is described.

Data. LOBSTER, Nasdaq, event-by-event, 4 January to 30 December 2016. Six stocks split by relative tick size: AMZN, TSLA, NFLX (small-tick) and ORCL, CSCO, MU (large-tick). Sessions 10:00–15:30. Each day is split into $I = 11$ non-overlapping 30-minute windows, each into $K = 180$ buckets of 10 seconds, giving one regression per window and $252 \times 11 = 2,772$ fits per stock.

3.3 Results

Table 1: Out-of-sample RMSE in ticks at $M = 10$, against best-level OFI. Improvements are as the paper reports them.

	AMZN	TSLA	NFLX	ORCL	CSCO	MU
OFI (best level)	9.72	5.35	2.03	0.25	0.19	0.22
MLOFI, OLS	8.53	4.97	1.49	0.09	0.06	0.09
MLOFI, ridge	8.05	4.53	1.41	0.08	0.05	0.08
Ridge improvement	17%	15%	31%	68%	74%	64%

Depth matters most where the tick is large: roughly 65–75% RMSE improvement for large-tick stocks against 15–30% for small-tick ones. Under OLS the out-of-sample error for AMZN and TSLA turns *upward* beyond about $M = 5$; under ridge it does not. That single contrast is the argument for regularising.

```
def mlofi(book, levels, bucket):
    """Raw MLOFI per level. No normalisation, per Xu et al."""
    grouper = pd.Grouper(freq=bucket, label="right", closed="right")
    cols = {}
    for m in range(1, levels + 1):
        bp, bs = book[f"bid_px_{m}"], book[f"bid_sz_{m}"]
        ap, asz = book[f"ask_px_{m}"], book[f"ask_sz_{m}"]
        bp0, bs0, ap0, as0 = bp.shift(1), bs.shift(1), ap.shift(1), asz.shift(1)
        dW = np.where(bp > bp0, bs, np.where(bp == bp0, bs - bs0, -bs0))
        dV = np.where(ap > ap0, -asz, np.where(ap == ap0, asz - as0, asz))
        cols[f"mlofi_{m}"] = pd.Series(dW - dV, index=book.index).groupby(grouper).sum()
    return pd.DataFrame(cols)

RIDGE_GRID = np.logspace(-5, 5, 50) # paper: 50 points in [1e-5, 1e5]

def fit_mlofi_ridge(feats, mid_change, window=180):
    return rolling_fit_predict(
        feats, mid_change, window=window,
        make_model=lambda: RidgeCV(alphas=RIDGE_GRID, cv=5),
    )
```

Listing 2: MLOFI with ridge, cross-validated as in the paper.

4 Integrated OFI and cross-impact

Source. Cont, Cucuringu & Zhang (2023), *Cross-Impact of Order Flow Imbalance in Equity Markets*, Quantitative Finance 23(10).

4.1 Definition

Per-event flows for stock i at level m :

$$\text{OF}_{i,n}^{m,b} = \begin{cases} q_{i,n}^{m,b}, & P_{i,n}^{m,b} > P_{i,n-1}^{m,b}, \\ q_{i,n}^{m,b} - q_{i,n-1}^{m,b}, & P_{i,n}^{m,b} = P_{i,n-1}^{m,b}, \\ -q_{i,n}^{m,b}, & P_{i,n}^{m,b} < P_{i,n-1}^{m,b}, \end{cases} \quad \text{OF}_{i,n}^{m,a} = \begin{cases} -q_{i,n}^{m,a}, & P_{i,n}^{m,a} > P_{i,n-1}^{m,a}, \\ q_{i,n}^{m,a} - q_{i,n-1}^{m,a}, & P_{i,n}^{m,a} = P_{i,n-1}^{m,a}, \\ q_{i,n}^{m,a}, & P_{i,n}^{m,a} < P_{i,n-1}^{m,a}. \end{cases} \quad (5)$$

Aggregating and scaling by average depth:

$$\text{OFI}_{i,t}^{m,h} = \sum_{n=N(t-h)+1}^{N(t)} (\text{OF}_{i,n}^{m,b} - \text{OF}_{i,n}^{m,a}), \quad \text{ofi}_{i,t}^{m,h} = \frac{\text{OFI}_{i,t}^{m,h}}{Q_{i,t}^{M,h}}, \quad (6)$$

$$Q_{i,t}^{M,h} = \frac{1}{M} \sum_{m=1}^M \frac{1}{2 \Delta N(t)} \sum_{n=N(t-h)+1}^{N(t)} (q_{i,n}^{m,b} + q_{i,n}^{m,a}). \quad (7)$$

One *common* denominator scales every level, which preserves the relative size of the levels. The motivation is the intraday pattern in depth: without it the same raw flow means different things at 10:00 and at 15:00.

The ten scaled levels are then compressed into one number:

$$\text{ofi}_{i,t}^{I,h} = \frac{\mathbf{w}_1^\top \mathbf{ofi}_{i,t}^{(h)}}{\|\mathbf{w}_1\|_1}, \quad \mathbf{ofi}_{i,t}^{(h)} = (\text{ofi}_{i,t}^{1,h}, \dots, \text{ofi}_{i,t}^{10,h})^\top. \quad (8)$$

The first principal component explains 89.06% (sd 6.12) of the variance of the level vector; a plain average across levels reaches 85.07%.

Under-specified step. The paper says only that $\mathbf{w}_1$ is “the first principal vector computed from historical data”. It does not state whether the PCA is on the correlation or the covariance matrix, nor the estimation window. Results are reported “averaged across each stock and trading day”, which suggests a per-stock, per-day fit, but the paper never states that as the rule. Fix your own choice and record it.

4.2 Models

With log mid-price returns $r_{i,t}^{(h)} = \log(P_{i,t}/P_{i,t-h})$:

$$\text{PI}^{[1]} : r_{i,t}^{(h)} = \alpha_i + \beta_i \text{ofi}_{i,t}^{1,h} + \epsilon_{i,t}, \quad \text{PI}^I : r_{i,t}^{(h)} = \alpha_i + \beta_i \text{ofi}_{i,t}^{I,h} + \epsilon_{i,t}, \quad (9)$$

$$\text{CI}^{[1]} : r_{i,t}^{(h)} = \alpha_i + \beta_{i,i} \text{ofi}_{i,t}^{1,h} + \sum_{j \neq i} \beta_{i,j} \text{ofi}_{j,t}^{1,h} + \eta_{i,t}, \quad \text{CI}^I : \text{same with } \text{ofi}_{i,t}^{I,h}. \quad (10)$$

The forecasting versions predict the next f -minute return from lagged aggregates over kh , $k \in L = \{1, 2, 3, 5, 10, 20, 30\}$:

$$\text{FPI}^{[1]} : r_{i,t+f}^{(f)} = \alpha_i + \sum_{k \in L} \beta_i^k \text{ofi}_{i,t}^{1,(kh)} + \epsilon_{i,t+f}, \quad (11)$$

$$\text{FCI}^{[1]} : r_{i,t+f}^{(f)} = \alpha_i + \sum_{j=1}^N \sum_{k \in L} \beta_{i,j}^k \text{ofi}_{j,t}^{1,(kh)} + \eta_{i,t+f}, \quad (12)$$

with $\text{ofi}_{i,t}^{I,(kh)}$ in place of $\text{ofi}_{i,t}^{1,(kh)}$ for the integrated variants FPI^I and FCI^I . The benchmarks are AR (own lagged returns) and CAR (all stocks' lagged returns), with the same lag set.

Estimation. PI and AR by OLS; CI, FCI and CAR by LASSO with a cross-validated ℓ_1 penalty (no fold count or grid stated, and no standardisation stated). LASSO is required because $N \approx 100$ regressors face 30 one-minute observations per window. Non-overlapping 30-minute windows within 10:00–15:30; one-minute buckets; physical time, not trade time; Nasdaq ITCH via LOBSTER, top 100 S&P 500 names, 2017–2019.

4.3 Results

Table 2: Contemporaneous R^2 in percentage points, and one-minute-ahead out-of-sample R^2 . Cross-impact adds 2.71 points in sample on best-level OFI but only 0.71 on integrated OFI, and nothing out of sample.

	PI ^[1]	CI ^[1]	PI ^I	CI ^I
In-sample, contemporaneous	71.16	73.87	87.14	87.85
Out-of-sample, contemporaneous	64.64	66.03	83.83	83.62
Out-of-sample, one minute ahead	−0.37	−0.10	−0.36	−0.10

For forecasting, the cross-sectional models beat their own-stock counterparts (AR −0.36, CAR −0.10), but integration brings no advantage: the PCA step discards which level the flow came from, and level identity carries predictive content. Annualised PnL under the paper’s trading rule, ignoring costs: 0.21 (FPI^[1]), 0.43 (FCI^[1]), 0.23 (FPI^I), 0.39 (FCI^I), 0.23 (AR), 0.40 (CAR).

The trading rule itself is worth stating, because both this paper and Sitaru et al. evaluate with it:

$$w_{i,t} = \frac{\mathbb{1}_{\{|f_{i,t}| > \text{sprd}_{i,t}\}} \cdot f_{i,t} / \sigma_{i,t}}{\sum_{n=1}^N \mathbb{1}_{\{|f_{n,t}| > \text{sprd}_{n,t}\}} \cdot f_{n,t} / \sigma_{n,t}}, \quad (13)$$

where $f_{i,t}$ is the model’s forecast for the next minute, $\text{sprd}_{i,t}$ the relative bid-ask spread, and $\sigma_{i,t}$ the standard deviation of the model’s in-sample forecasts over the previous 30 minutes. If no forecast exceeds its spread, nothing trades that minute.

```
def depth_scaled_ofi(book, levels, bucket):
    """Scaled level OFIs sharing one depth denominator  $Q \sim \{M, h\}$ ."""
    grouper = pd.Grouper(freq=bucket, label="right", closed="right")
    n_events = book.groupby(grouper).size()
    depth = []
    for m in range(1, levels + 1):
        depth.append((book[f"bid_sz_{m}"] + book[f"ask_sz_{m}"]).groupby(grouper).sum())
    Q = pd.concat(depth, axis=1).sum(axis=1) / (2.0 * levels * n_events)

    cols = {}
    for m in range(1, levels + 1):
        bp, bs = book[f"bid_px_{m}"], book[f"bid_sz_{m}"]
        ap, asz = book[f"ask_px_{m}"], book[f"ask_sz_{m}"]
        bp0, bs0, ap0, as0 = bp.shift(1), bs.shift(1), ap.shift(1), asz.shift(1)
        ofb = np.where(bp > bp0, bs, np.where(bp == bp0, bs - bs0, -bs))
        ofa = np.where(ap > ap0, -asz, np.where(ap == ap0, asz - as0, asz))
        raw = pd.Series(ofb - ofa, index=book.index).groupby(grouper).sum()
        cols[f"ofi_{m}"] = raw / Q
    return pd.DataFrame(cols)

from sklearn.decomposition import PCA

def integrated_ofi(scaled, fit_window):
```

```

"""Project onto the first PC of past scaled OFIs, l1-normalised weights.

fit_window rows ending before t are used, so the weight vector applied at t
never sees t. The paper does not state the window; record whatever you pick.
"""

out = pd.Series(index=scaled.index, dtype=float)
vals = scaled.to_numpy()
for t in range(fit_window, len(scaled)):
    past = vals[t - fit_window:t]
    w = PCA(n_components=1).fit(past).components_[0]
    out.iloc[t] = float(vals[t] @ w / np.abs(w).sum())
return out

```

Listing 3: Integrated OFI. The PCA is fitted on past data only; the window is a choice the paper leaves open, so it is a parameter here.

5 Decomposed OFI by event type

Source. Sitaru, Calinescu & Cucuringu (2023), *Order Flow Decomposition for Price Impact Analysis in Equity Limit Order Books*, ICAIF '23.

5.1 Definition

For event n at level ℓ ,

$$\text{aOF}_n^\ell = \begin{cases} -qa_n^\ell, & a_n^\ell > a_{n-1}^\ell, \\ qa_n^\ell - qa_{n-1}^\ell, & a_n^\ell = a_{n-1}^\ell, \\ qa_n^\ell, & \text{otherwise,} \end{cases} \quad \text{bOF}_n^\ell = \begin{cases} qb_n^\ell, & b_n^\ell > b_{n-1}^\ell, \\ qb_n^\ell - qb_{n-1}^\ell, & b_n^\ell = b_{n-1}^\ell, \\ -qb_{n-1}^\ell, & \text{otherwise,} \end{cases} \quad (14)$$

$$\text{OFI}_t^{\ell,h} = \sum_{n=N(t-h)+1}^{N(t)} (\text{bOF}_n^\ell - \text{aOF}_n^\ell), \quad Q_t^{L,h} = \frac{1}{L} \sum_{\ell=1}^L \frac{1}{\Delta N(t)} \sum_{n=N(t-h)+1}^{N(t)} \frac{qa_n^\ell + qb_n^\ell}{2}, \quad (15)$$

and $\text{ofi}_t^{\ell,h} = \text{OFI}_t^{\ell,h} / Q_t^{L,h}$.

The decomposition splits the same sum by event type:

$$\text{OFIa}_t^{\ell,h} = \sum_{n=N(t-h)+1}^{N(t)} (\text{bOF}_n^\ell - \text{aOF}_n^\ell) \mathbb{1}_{\{\text{type}(n)=\text{add}\}}, \quad (16)$$

with OFIc (cancel) and OFIt (trade) analogous, so that

$$\text{OFI}_t^{\ell,h} = \text{OFIa}_t^{\ell,h} + \text{OFIc}_t^{\ell,h} + \text{OFIt}_t^{\ell,h}. \quad (17)$$

Each component is divided by the same $Q_t^{L,h}$, so additivity survives normalisation. Event types need market-by-order data; they cannot be recovered from snapshots.

LOBSTER mapping. add = type 1; cancel = types 2 and 3; trade = types 4, 5 and 6. Execution of a hidden order does not change the book, so its contribution is null.

5.2 Models and estimation

$$\text{PI}^\ell : r_t^h = \alpha + \sum_{k=1}^{\ell} \beta_k \text{ofit}_t^{k,h} + \epsilon, \quad (18)$$

$$\text{PID}^\ell : r_t^h = \alpha + \sum_{k=1}^{\ell} (\beta_{k,\text{aofia}}^{k,h} + \beta_{k,\text{cofic}}^{k,h} + \beta_{k,\text{tfit}}^{k,h}) + \epsilon, \quad (19)$$

and forecasting versions FPI_M^ℓ , FPID_M^ℓ with lag sets $M_3 = \{1, 2, 3\}$ and $M_7 = \{1, 2, 3, 5, 10, 20, 30\}$ minutes. Contemporaneous: $h = 10$ s, 10 levels, 180-observation window, next 30 minutes out of sample. Forecasting: $h = f = 1$ min, rolling 30-observation window, LASSO with cross-validation except the two best-level OLS baselines. Same LOBSTER universe as Cont et al.

5.3 Results

Table 3: Contemporaneous adjusted R^2 , percentage points (sd in parentheses). Out-of-sample values for decomposed OFI under OLS at $\ell \geq 6$ fall below -100% and are not reported by the authors.

Model	In-sample	Out-of-sample
PI^1 (OLS)	73.67 (14.35)	69.42 (22.31)
PI^{10} (OLS)	91.97 (8.73)	89.37 (21.06)
PID^{10} (OLS)	93.00 (7.37)	not reported
PID^{10} (LASSO)	92.23 (8.82)	87.03 (14.85)
$\text{PID}^{I,10}$ (10 components)	90.57 (11.01)	86.24 (19.34)

Contemporaneously the decomposition adds nothing: about one point in sample, about two given back out of sample. The principal-component structure differs sharply from section 4: the first component of the 30-variable decomposed vector explains only 59.43% of its variance, and five components are needed for 89.24%. Event type genuinely adds dimensions.

Forecasting is where it pays. Every out-of-sample R^2 is negative — the best is $\text{FPI}_{M_3}^1$ at -0.082 — while the trading evaluation improves several-fold:

Table 4: Forecast-implied PnL in basis points per dollar traded, and Sharpe ratio, under the trading rule of section 4. Costs excluded.

	FPI^1 (OLS)	FPI^1 (LASSO)	FPID^1	FPI^{10}	FPID^{10}
M_3	0.22 / 1.24	0.59 / 2.17	1.01 / 4.26	0.54 / 2.09	1.66 / 6.90
M_7	0.68 / 5.86	1.26 / 5.22	2.42 / 10.65	1.33 / 5.18	3.15 / 13.51

Writing $\mathcal{F}(\cdot)$ for the LASSO selection frequency, at every level and lag

$$\mathcal{F}(\text{adds}) > \mathcal{F}(\text{cancels}) > \mathcal{F}(\text{trades}),$$

and in the forecasting model the trade component is rarely selected at all. Order submissions, not executions, carry the forecasting content. Caveat from the authors: $\text{FPID}_{M_7}^{10}$ has 210 regressors against 30 observations, and 53% of its regressions select nothing but the intercept, so the headline Sharpe ratio rests on a minority of active minutes.

```

ADD, CANCEL, TRADE = "add", "cancel", "trade"
LOBSTER_TYPE_MAP = {1: ADD, 2: CANCEL, 3: CANCEL, 4: TRADE, 5: TRADE, 6: TRADE}

def decomposed_ofi(book, levels, bucket):
    """Normalised OFI and its add/cancel/trade parts, per level and bucket."""
    etype = book["event_type"] # already mapped
    grouper = pd.Grouper(freq=bucket, label="right", closed="right")
    n_events = book.groupby(grouper).size()

    per_level = [((book[f"ask_sz_{l}"] + book[f"bid_sz_{l}"]) / 2.0)
                  .groupby(grouper).sum() for l in range(1, levels + 1)]
    Q = pd.concat(per_level, axis=1).sum(axis=1) / (levels * n_events)

    cols = {}
    for l in range(1, levels + 1):
        bp, bs = book[f"bid_px_{l}"], book[f"bid_sz_{l}"]
        ap, asz = book[f"ask_px_{l}"], book[f"ask_sz_{l}"]
        bp0, bs0, ap0, as0 = bp.shift(1), bs.shift(1), ap.shift(1), asz.shift(1)
        bof = np.where(bp > bp0, bs, np.where(bp == bp0, bs - bs0, -bs0))
        aof = np.where(ap > ap0, -as0, np.where(ap == ap0, asz - as0, asz))
        signed = pd.Series(bof - aof, index=book.index).fillna(0.0)
        cols[f"ofi_{l}"] = signed.groupby(grouper).sum() / Q
        for name in (ADD, CANCEL, TRADE):
            cols[f"ofi_{name}_{l}"] = signed.where(etype == name, 0.0).groupby(grouper).sum() / Q

    out = pd.DataFrame(cols).dropna(how="all")
    for l in range(1, levels + 1):
        parts = out[[f"ofi_{n}_{l}" for n in (ADD, CANCEL, TRADE)]]
        assert np.allclose(parts, out[f"ofi_{l}"], atol=1e-8, equal_nan=True), l
    return out

def lagged_blocks(feats, lags):
    """Aggregate each feature over m-bucket windows, one block per lag."""
    frames = []
    for m in lags:
        rolled = feat.rolling(window=m, min_periods=m).sum()
        rolled.columns = [f"{c}_lag{m}" for c in feat.columns]
        frames.append(rolled)
    return pd.concat(frames, axis=1)

M3, M7 = (1, 2, 3), (1, 2, 3, 5, 10, 20, 30)

def fit_forecast(feats, fwd_ret, lags=M7, window=30):
    X = lagged_blocks(feats, lags).dropna()
    y = fwd_ret.reindex(X.index)
    return rolling_fit_predict(
        X, y, window=window,
        make_model=lambda: make_pipeline(StandardScaler(),
                                          LassoCV(cv=3, n_alphas=25, max_iter=5000)),
    )

```

Listing 4: Decomposed OFI. The assertion enforces the additivity identity, which is the cheapest test that the event mapping is right.

6 Generalised OFI for snapshot data

Source. Su, Sun, Li & Yuan (2021), *The Price Impact of Generalized Order Flow Imbalance*, arXiv:2112.02947.

6.1 The problem and the fix

Standard OFI accounts for changes at the *position* of the best quote, which is exact only if the quote moves at most one tick between observations. Chinese exchanges publish snapshots every three seconds, over which the touch can traverse several levels. Generalised OFI keys on the *value* of the best price and sums the quantities at every level traversed:

$$\text{GOFI}_{k-1,k} = \sum_{n=1}^N \text{gene}_n, \quad \text{gene}_n = \Delta G\text{Bid}_n - \Delta G\text{Ask}_n, \quad (20)$$

$$\Delta G\text{Bid}_n = \begin{cases} \sum_{i \leq m^b} v_{n,i}^b - v_{n-1,1}^b, & P_n^b > P_{n-1}^b, \\ v_{n,1}^b - v_{n-1,1}^b, & P_n^b = P_{n-1}^b, \\ v_{n,1}^b - \sum_{i \leq m^b} v_{n-1,i}^b, & P_n^b < P_{n-1}^b, \end{cases} \quad \Delta G\text{Ask}_n = \begin{cases} v_{n,1}^a - \sum_{i \leq m^a} v_{n-1,i}^a, & P_n^a > P_{n-1}^a, \\ v_{n,1}^a - v_{n-1,1}^a, & P_n^a = P_{n-1}^a, \\ \sum_{i \leq m^a} v_{n,i}^a - v_{n-1,1}^a, & P_n^a < P_{n-1}^a, \end{cases} \quad (21)$$

where $v_{n,i}^b$ is the queue size at the i -th bid level at n , and the number of levels crossed is

$$m^b = \left\lceil \frac{P_n^b - P_{n-1}^b}{\delta} \right\rceil + 1, \quad m^a = \left\lceil \frac{P_n^a - P_{n-1}^a}{\delta} \right\rceil + 1, \quad (22)$$

with δ the tick size.

Stationarised variants. log-OFI and log-GOFI apply the logarithm to the *queue quantities* before differencing, not to the imbalance. The paper states this in words only and prints no display equation for log-OFI, so do not present one as theirs. The motivation is that queue sizes vary enormously across levels, which the fixed-depth assumption of the original model does not accommodate.

6.2 Results

Univariate regressions of the mid-price change on each indicator, at horizons of 30 seconds, 1 minute and 5 minutes; ten CSI 500 constituents; three-second snapshots; in-sample January to March 2021, out-of-sample April to June 2021. The paper gives no formula for how its R^2 values are computed, so fix and record your own (the harness in section 2 uses a zero benchmark).

Table 5: Average R^2 across the ten stocks, percentage points, in-sample / out-of-sample.

Horizon	OFI	log-OFI	GOFI	log-GOFI
30 seconds	38.38 / 32.89	75.96 / 76.37	45.73 / 40.35	83.58 / 83.57
1 minute	44.45 / 38.13	77.63 / 77.85	51.88 / 46.27	85.46 / 85.37
5 minutes	49.64 / 42.57	77.04 / 77.36	56.76 / 51.65	85.76 / 86.01

Three things stand out. The log transform buys more than the generalisation does, contrary to the paper’s framing. The two are close to additive. And for the generalised indicators the in-sample and out-of-sample values essentially coincide, where plain OFI loses five to seven points.

Read this one carefully. GOFI aggregates *resting depth across traversed levels* rather than incremental flow at a fixed level, so it is closer to a state variable of the book than to a flow variable. The paper does not discuss whether that mechanically narrows the gap to the contemporaneous mid-price change. Anyone building on it should check that separately.

```
def generalized_ofi(snaps, tick, n_levels=5, use_log=False):
    """GOFI / log-GOFI from L2 snapshots (Su et al.).

    'snaps' has bid_px_1..n, bid_sz_1..n, ask_px_1..n, ask_sz_1..n per snapshot.
    """
    def v(side, i, shift=0):
        s = snaps[f"{side}_sz_{i}"]
        s = s.shift(shift) if shift else s
        return np.log1p(s) if use_log else s

    bp, ap = snaps["bid_px_1"], snaps["ask_px_1"]
    bp0, ap0 = bp.shift(1), ap.shift(1)
    mb = (bp - bp0).abs().div(tick).round().astype("Int64").fillna(0) + 1
    ma = (ap - ap0).abs().div(tick).round().astype("Int64").fillna(0) + 1

    def cum(side, upto, shift):
        """Sum of the first 'upto' levels, row-wise, capped at n_levels."""
        acc = pd.Series(0.0, index=snaps.index)
        for i in range(1, n_levels + 1):
            acc = acc.add(v(side, i, shift).where(upto >= i, 0.0), fill_value=0.0)
        return acc

    d_bid = np.where(bp > bp0, cum("bid", mb, 0) - v("bid", 1, 1),
                     np.where(bp == bp0, v("bid", 1, 0) - v("bid", 1, 1),
                               v("bid", 1, 0) - cum("bid", mb, 1)))
    d_ask = np.where(ap > ap0, v("ask", 1, 0) - cum("ask", ma, 1),
                     np.where(ap == ap0, v("ask", 1, 0) - v("ask", 1, 1),
                               cum("ask", ma, 0) - v("ask", 1, 1)))
    return pd.Series(d_bid - d_ask, index=snaps.index, name="gofi")
```

Listing 5: Generalised OFI from snapshots, with the optional log transform.

7 OFI as a shock: horizon choice and screening

Source. Hu & Zhang (2025), *Stochastic Price Dynamics in Response to Order Flow Imbalance: Evidence from CSI 300 Index Futures*, arXiv:2505.17388.

7.1 Four metrics, stated in one place

$$\text{OFI}_k = \sum_{n=N(t_{k-1})+1}^{N(t_k)} e_n, \quad \text{TI}_k = \sum_{n=N(t_{k-1})+1}^{N(t_k)} \omega_n, \quad (23)$$

$$e_n = \mathbb{K}_{\{p_n^B \geq p_{n-1}^B\}} q_n^B - \mathbb{K}_{\{p_n^B \leq p_{n-1}^B\}} q_{n-1}^B - \mathbb{K}_{\{p_n^A \leq p_{n-1}^A\}} q_{n-1}^A + \mathbb{K}_{\{p_n^A \geq p_{n-1}^A\}} q_n^A, \quad (24)$$

$$\omega_n = \left(\mathbb{K}_{\{p_n^{\text{last}} > M_p\}} - \mathbb{K}_{\{p_n^{\text{last}} < M_p\}} \right. \\ \left. + \mathbb{K}_{\{p_n^{\text{last}} = M_p, p_n^{\text{last}} \geq p_{n-1}^{\text{last}}\}} - \mathbb{K}_{\{p_n^{\text{last}} = M_p, p_n^{\text{last}} < p_{n-1}^{\text{last}}\}} \right) v, \quad (25)$$

with $M_p = (P_n^A + P_n^B)/2$ and $v = V_n - V_{n-1}$ the incremental volume. The indicator form of e_n is the same object as the piecewise definitions above, written compactly.

$$\text{Lambda}_k = \frac{p_k^{\max} - p_k^{\min}}{N(t_k) - N(t_{k-1})}, \quad \text{AvgEn}_k = \text{OFI}_{N(t_k)}^{\text{avg}} - \text{OFI}_{N(t_{k-1})}^{\text{avg}}, \quad \text{OFI}_{N(t_k)}^{\text{avg}} = \frac{1}{N(t_k)} \sum_{n=1}^{N(t_k)} e_n. \quad (26)$$

7.2 The shock model

The point of the paper is that a fixed window and a matching forecast horizon are an arbitrary pairing. An OFI accumulated over a window is instead treated as a step shock whose response is measured across many horizons. The drift of geometric Brownian motion is replaced by a mean-reverting process driven by a jump Lévy process:

$$dS_t = \mu_t S_t dt + \sigma S_t dW_t, \quad d\mu_t = \theta(\mu_l - \mu_t) dt + dL_t, \quad \mu_0 = \text{OFI}_0 \rho k, \quad (27)$$

with μ_l the long-run drift (set to zero empirically), θ the mean-reversion speed and L_t a zero-mean symmetric Lévy process with $\sigma_L^2 = \mathbb{E}[L_t^2]/t$. The Lévy driver is an empirical choice: tick-level flow contributions are heavy-tailed, so a Gaussian driver misfits. For $R_t = \log(S_t/S_0)$,

$$\mathbb{E}[R_t] = \mu_0 \frac{1 - e^{-\theta t}}{\theta} - \frac{\sigma^2}{2} t, \quad \text{Var}(R_t) = \sigma^2 t + \frac{\sigma_L^2}{\theta^2} \left[t - \frac{2}{\theta} (1 - e^{-\theta t}) + \frac{1}{2\theta} (1 - e^{-2\theta t}) \right], \quad (28)$$

and the *response ratio* (their quasi-Sharpe ratio) is the ratio of the two,

$$QS(t) = \frac{\mathbb{E}[R_t]}{\sqrt{\text{Var}(R_t)}}. \quad (29)$$

It vanishes as $t \rightarrow 0^+$ like $\sqrt{t}$ and tends to a constant as $t \rightarrow \infty$. Practically it says where in horizon space a signal is worth trading: the numerator saturates while the denominator keeps growing.

7.3 A screening test worth reusing

The paper’s most portable contribution is an ex-ante test for keeping any microstructure indicator. A robust metric should

1. correlate significantly with contemporaneous price moves, with coefficients stable across time windows;
2. show memory — high autocorrelation with slow decay;
3. preserve both properties across market regimes, varying quantitatively but *never in sign*.

Sign reversal across regimes is the disqualifying failure.

Data and grid. CSI 300 index futures; exchange snapshots every 500 ms, so one “tick” is 500 ms; roughly six million ticks over December 2023 to November 2024. Historical windows of 1, 2, 5, 10, 20, 30, 60, 120 and 240 ticks are crossed against forecast horizons of 1 to 3600 ticks (30 minutes). LASSO with 5-fold cross-validation, in-sample to out-of-sample ratio 4:1.

```

def response_ratio(t, mu0, theta, sigma, sigma_L):
    """QS(t) for a step OFI shock (Hu & Zhang, eq. 24)."""
    t = np.asarray(t, dtype=float)
    mean = mu0 * (1.0 - np.exp(-theta * t)) / theta - 0.5 * sigma**2 * t
    var = sigma**2 * t + (sigma_L**2 / theta**2) * (
        t - (2.0 / theta) * (1.0 - np.exp(-theta * t))
        + (1.0 / (2.0 * theta)) * (1.0 - np.exp(-2.0 * theta * t)))
    return mean / np.sqrt(var)

def screen_indicator(feature, ret, regime_labels, max_lag=120):
    """The three criteria, as numbers you can threshold.

    Returns contemporaneous correlation, its dispersion across regimes, the
    autocorrelation decay, and whether the sign is stable across regimes.
    """
    corr_all = feature.corr(ret)
    by_regime = {r: feature[regime_labels == r].corr(ret[regime_labels == r])
                  for r in regime_labels.unique()}
    acf = [feature.autocorr(lag) for lag in range(1, max_lag + 1)]
    signs = {np.sign(v) for v in by_regime.values() if np.isfinite(v)}
    return {
        "corr": corr_all,
        "corr_by_regime": by_regime,
        "corr_dispersion": float(np.nanstd(list(by_regime.values()))),
        "acf_half-life": next((i for i, a in enumerate(acf, 1) if a < acf[0] / 2), np.nan),
        "sign_stable": len(signs) == 1, # criterion 3: the hard one
    }

```

Listing 6: Response-ratio curve and the regime-stability screen.

8 Generalised imbalance from trades alone

Sources. Maitrier & Bouchaud (2025), arXiv:2506.07711, and the simulation companion Maitrier, Loeper & Bouchaud (2025), arXiv:2509.05065.

This is the construction that ports to markets without a visible book, because it needs only signed trades and their sizes.

8.1 Definition

$$I_T^a = \sum_{t \in [0, T]} \varepsilon_t (q_t)^a. \quad (30)$$

The exponent a sets how much large trades count: $a = 0$ is pure sign imbalance, $a = 1$ is volume imbalance, and intermediate values interpolate. The companion paper writes the same object in integral form, $I_T^a = \int_0^T dt \varepsilon_t q_t^a$.

The framework around it starts from the square-root law,

$$I(Q) = Y \sigma \sqrt{\frac{Q}{\varphi}}, \quad (31)$$

with Q the metaorder volume, σ the volatility, Y an $O(1)$ constant and φ the average executed order flow per unit time. Metaorders arrive at rate ν with signs ε , durations drawn from $\Psi_q(s) =$

$\mu_q s_0^{\mu_q} / s^{1+\mu_q}$ and child orders executed at rate ϕ . Two dependencies do the work:

$$\mu_q = \mu_1 + \lambda \ell, \quad \beta_q = \beta_1 - \lambda' \ell, \quad \ell = \log q, \quad (32)$$

so larger child orders are less persistent and their impact decays more slowly, and a long-range correlation *between* metaorder signs, $\mathbb{E}[\varepsilon(t)\varepsilon(t+\tau)] \simeq \Gamma(\tau_0/\tau)^{\gamma_\times}$.

Diffusion condition. Transient impact with heavy-tailed durations alone gives sub-diffusion. Diffusive prices require

$$\gamma_\times = 1 - 2\beta, \quad (33)$$

which with $\beta = 0.2$ gives $\gamma_\times = 0.6$. This is the paper’s central falsifiable claim.

8.2 What to measure on data

The testable object is the correlation between returns and generalised imbalance,

$$R_a(T) = \frac{\mathbb{E}[\Delta_T I_T^a]}{\sqrt{\mathbb{E}[\Delta_T^2]} \sqrt{\mathbb{E}[(I_T^a)^2]}}, \quad (34)$$

as a function of a and T . Two predictions follow: the scaling exponent of $\mathbb{E}[\Delta_T I_T^a]$ against T is *non-monotonic* in a , and $R_a(T)$ is *hump-shaped* in a — there is an optimal weighting of large trades. Both are observed. Empirically the peak sits at $a^* \approx 0.5$ for LLOY, TSCO and SPMINI and $a^* \approx 1$ for EUROSTOXX, with peak correlations around 0.45 for the stocks, 0.6 for SPMINI and 0.75 for EUROSTOXX.

If a large share of volatility came from fundamental news instead, the covariance would be T -independent and free of a -dependence. It is not. That is the paper’s argument against the efficient-market reading.

Practical notes. Everything is measured in *trade time*: T counts trades, not seconds. Translating to real time is non-trivial because activity is intermittent and has a U-shaped intraday profile; the paper neglects both. Four assets: LLOY and TSCO (LSE, 2012–2015), SPMINI (2022) and EUROSTOXX (2016–2018). Trades beyond 1% of daily volume are clipped. On EUROSTOXX the sign imbalance collapses onto a master curve with $\chi = 0.72$.

8.3 The simulator, and why it matters for bonds

The companion paper simulates the model without the analytical approximations and recovers the same phenomenology. Its most useful result for anyone without metaorder labels: **proxy metaorders reconstructed from anonymous trades reproduce the square-root law, including its prefactor Y** . Two conditions are essential — preserving the exact trade history and sampling trades without replacement, and an inter-time threshold so that a long gap between child orders starts a new metaorder. Agreement degrades toward linear at small Q/V_D , because short proxy metaorders miss the true start, where impact is most concave. Code: <https://github.com/glatouille/ArtificialMarketSimulator.git>.

```
def generalized_imbalance(signs, sizes, a, window):
    """I^a_T over a rolling window of 'window' trades (trade time)."""
    contrib = np.sign(signs) * np.power(np.asarray(sizes, dtype=float), a)
    return pd.Series(contrib, index=signs.index).rolling(window).sum()
```

```

def r_a_curve(signs, sizes, log_mid, T, a_grid=np.linspace(0.0, 2.0, 21)):
    """R_a(T) against a. The peak is the optimal weighting of large trades."""
    fwd = log_mid.shift(-T) - log_mid # return over T trades
    rows = []
    for a in a_grid:
        imb = generalized_imbalance(signs, sizes, a, T)
        both = pd.concat([fwd, imb], axis=1).dropna()
        num = (both.iloc[:, 0] * both.iloc[:, 1]).mean()
        den = np.sqrt((both.iloc[:, 0] ** 2).mean() * (both.iloc[:, 1] ** 2).mean())
        rows.append({"a": a, "R_a": num / den if den > 0 else np.nan})
    out = pd.DataFrame(rows)
    return out, float(out.loc[out["R_a"].idxmax(), "a"]) # a*, the hump location

def proxy_metaorders(trades, phi, C=4.0):
    """Group anonymous trades into proxy metaorders, per the companion paper.

    Conditions: same sign, no reuse of a trade, and a gap threshold C/phi
    between consecutive child orders.
    """
    out, current, last_t = [], [], None
    gap = C / phi
    for t, row in trades.iterrows():
        if current and (row["sign"] != current[-1]["sign"] or (t - last_t) > gap):
            out.append(current)
            current = []
        current.append({"t": t, **row.to_dict()})
        last_t = t
    if current:
        out.append(current)
    return out

```

Listing 7: Generalised imbalance and the hump in a . This runs on a trade tape alone: signs, sizes, prices.

9 Building flow features by clustering

Source. Petit, Cucuringu & Cartea (2025), *Data-Driven Trade Flow Decomposition for Exchange-Traded Funds and their Constituents*, ICAIF '25.

This is the template for turning raw flow into features that a downstream model can use. The pipeline is: build features around each trade, normalise them against comparable history, reduce dimension, cluster, then form cluster-conditional imbalances.

9.1 The 16 features

Around a central trade (i, n) , count the trades and sum the notional in a window $\pm\tau$, split three ways: same or opposite direction, ETF or constituents, before or after. That gives eight cells, each yielding a count and a notional.

All sixteen share one shape. For a market set $\mathcal{S}$ (the ETF $\mathcal{E}$ or its constituents $\mathcal{M}$), a direction

Table 6: Feature indices. Each cell contributes a trade count and a notional.

Direction	Market	Count	Notional
Same	ETF	1	2
Same	ETF (after)	3	4
Same	Constituents	5	6
Same	Constituents (after)	7	8
Opposite	ETF	9	10
Opposite	ETF (after)	11	12
Opposite	Constituents	13	14
Opposite	Constituents (after)	15	16

condition D (same or opposite to the central trade) and a window side W (before or after),

$$x_{i,n} = \sum_{j,m} w_{j,m} \cdot \llbracket j \in \mathcal{S} \rrbracket \cdot \llbracket D(\varepsilon_{j,m}, \varepsilon_{i,n}) \rrbracket \cdot \llbracket t_{j,m} \in W(t_{i,n}, \tau) \rrbracket, \quad (35)$$

where $w_{j,m} = v_{j,m} p_{j,m}$ gives the notional feature and $w_{j,m} = 1$ the trade count, with v the quantity, p the price, ε the direction and $\llbracket \cdot \rrbracket$ the Iverson bracket. “Before” is $t_{j,m} \in (t_{i,n} - \tau, t_{i,n})$ and “after” is $t_{j,m} \in (t_{i,n}, t_{i,n} + \tau)$. The calibrated window is $\tau^* = 707 \mu\text{s}$, fitted on 2017 SPY data.

9.2 Normalisation, reduction, clustering

Percentile rank against comparable history. Each feature is replaced by its rank within a 90-day lookback restricted to the *same time-of-day bucket* (65-minute buckets, six per session):

$$\tilde{x}_{i,n}^{(j)} = \frac{\sum_{m \in \mathcal{W}_{i,n}} \llbracket x_{i,m}^{(j)} < x_{i,n}^{(j)} \rrbracket}{|\mathcal{W}_{i,n}| - 1} \in [0, 1]. \quad (36)$$

This is the step that makes a microstructure feature comparable across days and across times of day, and it is the one most often skipped.

PCA. On mean-centred normalised features (centred, not variance-standardised); M is the smallest number of components with at least 90% explained variance, calibrated to $M^* = 5$. Components are sign-aligned across windows, $v_{j,d+1} \leftarrow \text{sgn}(v_{j,d+1}^\top v_{j,d}) v_{j,d+1}$, without which cluster labels flip arbitrarily.

Clustering. Mini-batch k -means++ on the component scores in a 14-day sliding window advancing daily: 100 initialisations, at most 100 passes, mini-batches of 2^{15} trades, lowest inertia kept, assignments retained only for the last day. Both the Calinski–Harabasz score and the Davies–Bouldin index select $K^* = 2$. Labels are matched across consecutive windows by

$$\pi_{d+1}^* = \arg \min_{\pi \in \text{Sym}(K)} \sum_{k=1}^K \|m_{\pi(k),d+1} - m_{k,d}\|^2. \quad (37)$$

Cluster-conditional order imbalance. With $V^\pm$ the buy and sell volumes,

$$\text{OI}_{i,d} = \frac{V_{i,d}^+ - V_{i,d}^-}{V_{i,d}^+ + V_{i,d}^-}, \quad (38)$$

and $COI_{i,k,d}$ the same using only trades in cluster k . These are orthogonalised against a seven-factor loading matrix (Fama–French five, UMD, and plain OI) and turned into factor-mimicking portfolios by minimum-variance weights subject to unit exposure to the target and zero to the rest.

Results. SPY and constituents, 2018–2021. At $K = 2$ the first cluster’s factor-mimicking portfolio earns an annualised Sharpe ratio of 1.91, significant at 1% after Bonferroni correction; the second earns -0.69 . At $K = 5$ only cluster 3 is significant at 5% (1.52). The claim to take away is narrow: *some* clusters carry information beyond raw order imbalance.

```
from sklearn.cluster import MiniBatchKMeans
from itertools import permutations

def percentile_normalise(feats, bucket_of_day, lookback_days=90):
    """Rank each feature within the same time-of-day bucket over 90 days."""
    out = pd.DataFrame(index=feats.index, columns=feats.columns, dtype=float)
    for b, idx in feats.groupby(bucket_of_day).groups.items():
        sub = feats.loc[idx].sort_index()
        win = f"{lookback_days}D"
        out.loc[idx] = sub.rolling(win, closed="left").apply(
            lambda w: (w[:-1] < w[-1]).sum() / max(len(w) - 1, 1), raw=True)
    return out

def align_components(new_v, old_v):
    """Sign-align PCs so cluster labels do not flip between windows."""
    return np.array([np.sign(n @ o) * n for n, o in zip(new_v, old_v)])

def match_labels(new_centres, old_centres):
    """Permutation of new cluster labels closest to the previous window."""
    best, best_cost = None, np.inf
    for perm in permutations(range(len(new_centres))):
        cost = sum(np.sum((new_centres[p] - old_centres[k]) ** 2)
                   for k, p in enumerate(perm))
        if cost < best_cost:
            best, best_cost = perm, cost
    return best

def cluster_conditional_oi(trades, labels, k):
    """COI for cluster k: signed volume share within that cluster."""
    sub = trades[labels == k]
    vp = sub.loc[sub["sign"] > 0, "volume"].sum()
    vm = sub.loc[sub["sign"] < 0, "volume"].sum()
    return (vp - vm) / (vp + vm) if (vp + vm) > 0 else np.nan
```

Listing 8: The Petit pipeline: percentile normalisation, PCA with sign alignment, mini-batch k-means with label matching, cluster-conditional imbalance.

10 Prerequisites for corporate bonds

Two things must be in place before any of the above is computed on a bond tape: the trades must be signed, and the prices must be corrected for bid-ask noise.

10.1 Signing trades

Source. Fedenia, Ronen & Nam (2021), *Machine Learning in the Corporate Bond Market: A New Classifier*, SSRN 3848068.

Every imbalance feature in this document needs ε_t . On TRACE the sign is not published in the standard feed, so it has to be inferred. The classical rules are:

- **Tick rule.** Buy if the previous price was lower, sell if higher; on a zero tick, compare with the nearest previous non-zero tick.
- **Quote rule.** Buy above the quote midpoint, sell below.
- **Lee–Ready.** Tick rule at the midpoint, quote rule outside it.
- **Bulk volume classification.** Aggregate into bars and split volume by $\hat{V}_\tau^B = V_\tau Z((P_\tau - P_{\tau-1})/\sigma_{\Delta P})$ with Z a Student- t CDF.

The paper trains a random forest on labelled TRACE Enhanced data, where the true initiator is known, and beats all of them: accuracy 81.1% for the best feature set against 68.8% for the tick rule across roughly 143 million trades, July 2002 to December 2019. On equities the same approach reaches 77.6% against 74.0% for Lee–Ready. The model is deliberately plain — 100 trees, one leaf — fitted on fifteen rolling three-year windows and tested on the following one and three years, which is the time-ordered split that matters.

Feature groups that carry the accuracy: price and five lags; time of day decomposed into hour, minute, second, weekday, month, quarter; volume and five lags; the tick-rule indicator itself as an input; running and rolling statistics over the last 20 trades; elapsed time since the last trade and since the last price change; and, where available, quotes.

Implementation warning. The labelled indicator exists in the Enhanced historical file, not in the standard feed. A classifier trained on one and applied to the other must respect that the feature set differs.

10.2 Correcting for bid-ask noise

Source. Dickerson, Robotti & Rossetti (2024), *Common Pitfalls in the Evaluation of Corporate Bond Strategies*, SSRN 4575879.

A price-based signal and the return that follows it share a price. Bid-ask bounce in that shared price creates predictability that is pure arithmetic. The correction is an *implementation gap*: build the signal from a price at least one trading day before the price that starts the return.

With $p_{t,d}$ the price on the first available trading day of month t and $p_{t,d+n}$ a price at least one trading day before the month-end price P_t ,

$$STREV_{i,t}^* = \frac{p_{i,t,d+n} + c_{i,t}}{p_{i,t,d}} - 1 \quad \text{against} \quad STREV_{i,t}^{MMN} = \frac{P_{i,t} + c_{i,t}}{P_{i,t-1}} - 1, \quad (39)$$

where $c_{i,t}$ is the coupon and all prices include accrued interest. The realised return is

$$r_{i,t+1}^B = \frac{P_{i,t+1}^f + C_{i,t+1}}{P_{i,t}^f} - 1, \quad (40)$$

with P^f the full price. The month-end price must fall in the last five business days of the month, and extending the gap beyond one business day changes little.

Effect. The value-weighted short-term reversal premium falls from 0.90% per month ($t = 4.79$) to 0.03%, an insignificant residue — a reduction over 90%. Alphas of other price-based sorts fall by 38% to 68%.

Ex ante versus ex post. Ex-post filtering sets the universe using a percentile of the pooled return distribution across all bonds and months, which uses information from after formation. Ex-ante filtering uses only information available at or before formation month t . The difference is not cosmetic: the Sharpe ratio of MOM(3,3) quintuples from 0.08 to 0.54 under the infeasible version, and MOM(6,6) goes from -0.06% per month feasible to 0.33% infeasible.

Corrected data and the construction code are public: <https://openbondassetpricing.com> and the PyBondLab package.

```
def implementation_gap_signal(prices, coupons, min_gap_days=1):
    """Signal price must precede the return's starting price by >= 1 trading day.

    'prices' is a per-bond daily series with a DatetimeIndex.
    """
    monthly = prices.resample("ME")
    first_px = monthly.first() #  $p_{\{t,d\}}$ 
    signal_px = prices.shift(min_gap_days).resample("ME").last() #  $p_{\{t,d+n\}}$ 
    ret_start = monthly.last() #  $P_t$ , starts the realised return
    signal = (signal_px + coupons.resample("ME").sum()) / first_px - 1.0
    # ret_start is returned so the caller can check that the signal price is
    # strictly earlier than the price that starts the realised return.
    return signal.rename("strev_corrected"), ret_start.rename("return_start_px")

def bond_flow_features(trades, bucket="1D"):
    """Trade-only imbalance features for a bond tape.

    Requires a signed tape (see section on signing). Uses the generalised
    imbalance of section 7, which needs no order book.
    """
    g = trades.groupby([pd.Grouper(freq=bucket), "isin"])
    vol = g["volume"].sum()
    signed = g.apply(lambda d: float((np.sign(d["sign"]) * d["volume"]).sum()))
    out = pd.DataFrame({"signed_volume": signed, "volume": vol})
    out["oi"] = out["signed_volume"] / out["volume"].replace(0.0, np.nan)
    for a in (0.0, 0.5, 1.0):
        out[f"imb_a{a}"] = g.apply(
            lambda d, a=a: float((np.sign(d["sign"]) * d["volume"] ** a).sum()))
    return out
```

Listing 9: Implementation gap and a signed bond tape.

11 Supporting results

Long memory of order flow. Gould, Porter & Howison (2016) estimate the Hurst exponent of order-sign series in FX spot at $H \approx 0.7$ across EUR/USD, GBP/USD and EUR/GBP, using detrended fluctuation analysis and log-periodogram regression, with $H = 1 - \gamma/2$ linking it to the

autocorrelation decay exponent. **Their sign convention is inverted:** +1 denotes a sell. They report H only, not γ .

Cross-response between stocks. Wang & Guhr (2018) define

$$R_{ij}(\tau) = \langle r_i(t, \tau) \varepsilon_j(t) \rangle_t, \quad i \neq j, \quad (41)$$

with r_i the log mid-price return and $\varepsilon_j \in \{-1, 0, +1\}$ the trade sign, signed from price changes rather than by Lee–Ready. There is no normalisation by volume in the definition. The finding relevant here: self-impact has a permanent component of order 10^{-11} against a temporary component of order 10^{-4} , and cross-impacts have *no* permanent component. Volume impact scales as $v^{0.51}$.

Fill probabilities. Lokin & Yu (2024) model limit order fill probabilities with state-dependent intensities, conditioning on the distance to the opposite best quote and the spread. Despite the common citation, **this paper defines no queue-imbalance variable:** it states only that imbalance is implicit in conditioning on both sides’ queue sizes. If you need a queue-imbalance definition, it has to come from elsewhere.

12 Source map

Table 7: What each paper contributes, and what it needs.

Paper	Contribution	Data required
Xu, Gould & Howison (2020)	Multi-level OFI vector; ridge over OLS	Full-depth book events
Cont, Cucuringu & Zhang (2023)	Depth scaling; integrated OFI via PCA; cross-impact	Full-depth book events, many names
Sitaru, Calinescu & Cucuringu (2023)	Add/cancel/trade decomposition	Market-by-order
Su et al. (2021)	Generalised OFI for snapshots; log transform	L2 snapshots
Hu & Zhang (2025)	Shock/horizon framing; indicator screening test	Snapshots, one instrument
Maitrier & Bouchaud (2025) + companion	Generalised imbalance I^a ; hump in a ; proxy metaorders	Signed trade tape only
Petit, Cucuringu & Cartea (2025)	16-feature construction; percentile normalisation; clustering	Trade tape, ETF + constituents
Fedenia, Ronen & Nam (2021)	Trade signing by random forest	Labelled TRACE Enhanced
Dickerson et al. (2024)	Implementation gap; ex-ante filtering	Bond tape with accrued interest
Gould et al.; Wang & Guhr; Lokin & Yu	Long memory; cross-response; fill probabilities	Various

What transfers to a bond tape. Sections 3 through 7 need a limit order book and do not transfer directly. Section 8 needs only signed trades and sizes, and section 9 needs only a trade tape plus a grouping structure. Those two are the portable ones, and both depend on section 10: sign the trades first, and put a gap between the signal price and the return price.